Iroko Historical Society
Iroko Framework Technical Series

Volume 1

From Source Text to Structured Dataset

PDF Parsing, Data Extraction, and Dataset Construction

This volume precedes:
Volume 2 — From Spreadsheet to Linked Data (OpenRefine RDF Transformation)
Volume 3 — The Mapping Workbook
Volume 4 — GitHub Pages and Ontology Site Maintenance

Iroko Historical Society | irokosociety.org | March 2026

Introduction

This manual documents the complete workflow for extracting structured ethnobotanical data from source PDFs and assembling it into the dataset that feeds the Iroko Framework RDF transformation pipeline. It is written for Iroko staff and interns with no prior experience in PDF parsing or Python, and for anyone who must reproduce, extend, or troubleshoot the dataset construction process at a future date.

The work described here is hands-on, iterative, and involves judgment calls that cannot be fully automated. An intern working through this process for the first time should expect to make several passes over the data before the output is reliable. That is not a failure of method — it is the nature of extracting structured data from digitized scholarly texts.

This volume covers two source texts:

- **Pierre Fatumbi Verger,** *Ewé: The Use of Plants in Yoruba Society* (1995, Odebrecht Editora, São Paulo). The primary botanical and Yoruba-language authority for the dataset. Contains a scientific glossary mapping scientific names to Yoruba names, and an extensive pharmacopeia of plant formulas organized by ritual and medicinal function.
- **Dalia Quiros-Moran,** *Guide to Afro-Cuban Herbalism.* The supplementary source for Spanish, English, Cuban, Lucumi, Haitian, and other regional vernacular names, Orisha associations, and ritual use context in the Afro-Atlantic diaspora.

Verger provides botanical depth and Yoruba linguistic authority. Quiros-Moran provides the diaspora linguistic and ritual bridge. Neither source alone is sufficient for the full dataset. Future volumes — the Brazilian Portuguese manuscript, fieldwork data from Irete Obara, Cabrera, and Font — will follow the same general workflow documented here, with source-specific adaptations covered in Part 7.

A Production QC Checklist is provided as Appendix B. It should be completed and signed off by the Executive Director before any dataset is considered ready for the transformation pipeline.

Part 1: Understanding Your Source Texts Before You Begin

Why source text structure determines everything

The single most important rule in any PDF extraction project: do not write a single line of parsing code until you understand the physical layout of the source document. Both the Verger and Quiros-Moran PDFs are digitized scans of print books. They are not natively digital documents. This distinction drives every decision that follows.

Scanned PDFs have no semantic structure. There are no headings, no tables, no tagged elements. Everything is a position on a page image processed by optical character recognition (OCR). The following problems are intrinsic to scanned PDFs and cannot be solved without understanding them first:

- Scanned PDFs carry no semantic structure — no headings, tables, or tagged elements. Everything is a position on a page image.
- OCR introduces errors. Older or more degraded scans have more errors. Diacritical marks, ligatures, and non-Latin characters are especially vulnerable.
- Multi-column layouts cause text extraction tools to merge columns incorrectly, producing output that interleaves left-column and right-column content unpredictably.
- Page headers, page numbers, footnotes, and captions get mixed into the main text stream.

Before writing any code, know the answers to these questions about your source:

- Is the text single-column or multi-column? Which sections use which layout?
- How are entries delimited — by whitespace, by a consistent header pattern, by page breaks?
- Are special characters (diacritics, tone marks) correctly encoded in the PDF, or have they been lost to OCR?
- Are there section headers, page numbers, or running titles that will contaminate the text stream?
- Is there a glossary, index, or other structured section that may be more parseable than the main body?

The Verger PDF: structure and challenges

The Verger PDF (Ewé) is a large document of several hundred pages with two distinct zones relevant to the dataset.

The scientific glossary (pages approximately 615–720)

This is the most structured and most useful section for extraction. The glossary lists scientific names with corresponding Yoruba names. The format on any given page is:

```
Egigun       -CEIBA PENTANDRA (L.) Gaertn., Bombacaceae
             ...(additional Yoruba names for same species)

Igi-odan     -CHLOROPHORA EXCELSA Benth & Hook., Moraceae
```

Extraction is complicated by several factors discovered during the parsing iterations for this dataset:

- **Two-column layout.** When pdftotext or pdfplumber extracts text without layout-aware settings, the two columns merge. Yoruba names from the left column appear on the same line as scientific names from the right column, or they interleave in unpredictable order.
- **OCR corruption at entry starts.** The leftmost characters at page margins are frequently the most damaged. A scan of "ABELMOSCHUS ESCULENTUS" may appear as ",−\BELMOSCHUS ESCULENTIJS". Do not assume the first character of a scientific name is reliable.
- **Species author citations.** Scientific names are followed by authorship and date notation in standard botanical convention (e.g., "(L.) Gaertn."). These must be separated from the binomial species name for reconciliation purposes.
- **Multiple Yoruba names per species.** Many plants have several Yoruba names spanning multiple lines, with the scientific name appearing only once.
- **Diacritic loss.** Yoruba tone marks and underdot characters (ẹ, ọ, ṣ) are frequently lost in OCR. The scanned version may show plain "e", "o", "s" where the print original has marked vowels. This cannot be corrected programmatically.

The pharmacopeia (main body)

The main body of Verger is organized as numbered formulas (oogun), each describing a plant or combination of plants for a specific ritual or medicinal purpose. The structure per formula is: formula number, Yoruba name, description of use, scientific name and family (sometimes), ethnographic notes. This section is far less structured than the glossary and far more prone to OCR errors. For the Verger dataset, the glossary was the primary extraction target. The pharmacopeia was consulted for supplementary context but was not systematically parsed in the current version of the dataset.

The Quiros-Moran PDF: structure and challenges

The Quiros-Moran guide has a more consistent entry format than Verger's main body, but its physical layout presented its own challenges.

Entry format

Each plant entry follows a consistent pattern:

```
COMMON NAME / ENGLISH NAME
(Scientific name)

Cuba:
  [Cuban common names]
Lukumi:
  [Lucumi names]
Kongo:
  [Kongo names]
Haiti:
  [Haitian Creole names]
Brazil:
  [Brazilian Portuguese names]
...(other regional variants: Colombia, Mexico, Venezuela, etc.)

[Description paragraph]

Medicinal uses:
  [Description]

Orisha/Belongs to:
  [Orisha name]
Correlating Odu:
  [Odu name]
```

Country list expansion

The Quiros-Moran source covers a large number of regional variants. The full country list that must be recognized by any parsing script is:

```
Cuba, Lukumi, Kongo, Haiti, Brazil, Colombia, Costa Rica,
El Salvador, Guatemala, Honduras, Mexico, Nicaragua,
Panama, Puerto Rico, US, Venezuela
```

When writing or updating a Quiros-Moran parser, this list must be included as a recognized section header set. Parsers written against a partial list will silently drop data from unrecognized countries into the wrong field.

> **WARNING:** *Do not assume the country list is exhaustive. If a new Quiros-Moran edition or a related source adds regional variants not in the above list, the parser will silently misclassify that data. Always verify the country header list against the actual source before running a full extraction.*

The two-column layout problem

The original Quiros-Moran PDF has a two-column page layout. When pdftotext extracts text without layout-aware settings, content from both columns interleaves — the left column content for one entry mixes with the right column content of the adjacent entry on the same page. The resulting text stream is incoherent and cannot be parsed reliably.

The solution is physical preprocessing before any code is run:

1. Open the Quiros-Moran PDF in a PDF editor that supports page cropping (Adobe Acrobat, PDF Expert, PDF24, or Smallpdf).
2. Identify the approximate horizontal midpoint of the two-column layout (typically around 50% of page width).
3. Create two versions: one cropped to the left half, one cropped to the right half.
4. Merge the left-column PDF and right-column PDF into a single file, with all left-column pages first.
5. Extract text from this merged single-column PDF. The resulting text stream will be coherent and parseable.

> **WARNING:** *Do not attempt to parse a two-column PDF directly. The interleaving problem is not reliably solvable in Python without either layout-aware bounding box extraction or the preprocessing step above. Attempting to parse the raw stream will produce entries that mix data from different plants.*

Part 2: Tool Selection and What Each Tool Does

pdftotext

pdftotext is a command-line tool from the Poppler library, available on Linux and macOS. It is the fastest option for plain text extraction when layout information is not critical. Install via Homebrew on macOS (brew install poppler) or apt on Linux (apt install poppler-utils).

```
# Basic extraction to a text file
pdftotext yourfile.pdf output.txt

# Extract to stdout (pipe into other tools)
pdftotext yourfile.pdf -

# Extract specific pages
pdftotext -f 615 -l 720 yourfile.pdf -

# Layout-preserving mode (attempts to maintain column positions)
pdftotext -layout yourfile.pdf -

# Layout mode for a single page (diagnostic use)
```

```
pdftotext -f 296 -l 296 -layout yourfile.pdf -
```

Whether to use plain mode or -layout mode requires testing on the specific document. For the Verger glossary, -layout mode improved column separation in some sections and worsened it in others. Test both before committing to a pipeline.

pdfplumber (Python)

pdfplumber is a Python library that provides control over PDF extraction, including the ability to extract text from specific bounding boxes. This is the correct tool when you need to isolate a specific column or region without preprocessing the file.

```
pip install pdfplumber --break-system-packages

import pdfplumber

with pdfplumber.open('yourfile.pdf') as pdf:
    page = pdf.pages[295]          # 0-indexed
    text = page.extract_text()

    # Or extract from a bounding box (left column only):
    region = page.within_bbox((0, 0, 300, page.height))
    left_text = region.extract_text()
```

pdfplumber is slower than pdftotext but gives programmatic control. Use it when pdftotext produces unusable output for a specific section.

PyMuPDF (fitz)

PyMuPDF (imported as fitz) handles complex layouts and non-Latin character encoding better than pdfplumber in many cases. It was used in the Verger project to inspect specific pages when other tools produced garbled output, and is the recommended tool for page-level diagnosis.

```
pip install pymupdf --break-system-packages

import fitz

doc = fitz.open('yourfile.pdf')
page = doc[295]                    # 0-indexed
print(page.get_text()[:500])
```

Use PyMuPDF for inspection and diagnosis. When pdftotext or pdfplumber gives strange output for a specific page, load that page in PyMuPDF and compare.

PyPDF2 / pypdf

PyPDF2 was the initial tool used in early Verger extraction attempts. It is functional but has less robust text extraction than pdfplumber or PyMuPDF for scanned documents. It is not recommended as the primary tool for this workflow. If you encounter it in older scripts in the project repository, it can be replaced with pdfplumber without changing the surrounding logic.

Part 3: The Iterative Parsing Process

Why it takes multiple passes

No PDF extraction of a scanned scholarly text produces clean output on the first attempt. The Verger dataset required multiple distinct parsing iterations before the output was reliable. Each iteration revealed a new class of problems that the previous approach had not handled. This is normal and expected. The general pattern is:

- Pass 1: Extract raw text and understand actual structure. Discover which tools and settings work for this document.
- Pass 2: Write a first parser. It will work for some entries and fail on others. Run it and examine failures.
- Pass 3: Fix the failure cases discovered in Pass 2. New failure cases will emerge.
- Pass 4+: Continue refining. For the Verger dataset, the primary extraction and cross-referencing work involved approximately five to seven distinct parsing iterations.

> **WARNING:** *Do not treat the first working pass as the final one. The most dangerous output is a parser that silently drops records — everything looks fine until you count the records and notice 196 plants are missing.*

Step 3.1 — Initial structure inspection

Before writing any parsing code, use the command line to examine the raw output of your extraction tool. Spend time here. The most common beginner mistake is to write a parser for an assumed format that does not match the actual format of the extracted text.

```
# See the first 200 lines of extracted text
pdftotext yourfile.pdf - | head -200

# Look at the glossary section specifically
pdftotext -f 615 -l 625 yourfile.pdf - | head -100

# Search for a known entry to verify it is present
pdftotext yourfile.pdf - | grep -i "ceiba pentandra"

# See lines surrounding a known entry
pdftotext yourfile.pdf - | grep -B5 -A5 "CEIBA"
```

```
# Check a specific page with layout mode
pdftotext -f 296 -l 296 -layout yourfile.pdf -
```

> **NOTE:** *The Verger glossary page structure was only understood correctly after using grep to find a known entry (Ceiba pentandra) and examining the surrounding lines. The actual format — Yoruba name on the left, scientific name after a dash, family after a comma — was not what was initially assumed from the document description.*

Step 3.2 — Write a first parser for a small sample

Do not run your parser on the full document first. Run it on 20–30 entries from a middle section of the glossary where scan quality is likely to be good. This gives fast feedback without waiting for a full-document parse.

For the Verger glossary, the working parsing pattern was:

```
import re

# Pattern: Yoruba_name -GENUS SPECIES Author, Family
# Example: Egigun -CEIBA PENTANDRA (L.) Gaertn., Bombacaceae

entry_pattern = re.compile(
    r'^([A-Za-z\u00e0-\u024f\s\(\)]+?)'  # Yoruba name
    r'\s+-'                                  # dash separator
    r'([A-Z][A-Z]+\s+[A-Z]+)'               # GENUS SPECIES in caps
    r'([^,]*)'                                 # author citation
    r',\s*([A-Za-z]+aceae)'                  # Family name
)
```

Run this on your sample, print what it matches and what it misses, and diagnose the misses before expanding to the full document.

Step 3.3 — Diagnose silently missing records

The most important diagnostic check is: how many records did I expect versus how many did I get? For the Verger glossary, the expected number can be estimated from page count and average entries per page. If your parser returns 77 records but you expected 273, something is wrong at a fundamental level.

```
# Build a list of species you know should be present
expected = ['Ceiba pentandra', 'Abelmoschus esculentus', 'Allium cepa']

# Check each expected entry against extracted results
extracted_names = [r['scientific_name'] for r in results]

for name in expected:
    found = any(name.lower() in n.lower() for n in extracted_names)
    if not found:
```

```
        print(f'MISSING: {name}')

# Always print a count summary
print(f'Extracted: {len(results)} records')
print(f'Expected:  ~273')
if len(results) < 200:
    print('WARNING: significant records may be missing')
```

When Abelmoschus esculentus was found to be missing from the Verger extraction, the diagnosis revealed an OCR error: the entry appeared as ",−\BELMOSCHUS ESCULENTIJS" rather than "ABELMOSCHUS ESCULENTUS". The initial regular expression expected entries to start with a capital letter and did not match entries with leading punctuation corruption. The fix was to relax the starting anchor condition and add a cleaning step that stripped leading punctuation before applying the main pattern.

Step 3.4 — Common OCR error patterns and how to handle them

First-character corruption

The leftmost characters at page margins are frequently damaged. Leading commas, backslashes, pipes, or tildes appearing before the first letter of a word are OCR artifacts. Strip them:

```
import re
# Remove leading non-alphabetic junk before processing
clean = re.sub(r'^[^A-Za-z]+', '', raw_entry)
```

I/J confusion in all-caps Latin names

In older typesetting, the letter J was used where modern convention uses I. OCR sometimes reverses this in unpredictable ways. ESCULENTIJS should be ESCULENTUS. When a scientific name fails reconciliation against Global Names or GBIF, check for this pattern before assuming the name is genuinely absent from the source.

Lost diacritics in Yoruba names

Yoruba orthography uses three types of marks frequently lost in scanned text: tone marks (acute, grave, macron over vowels), underdot (ẹ, ọ) and underdot-with-tone (ẹ́, ọ́), and s-with-dot-below (ṣ). When a Yoruba name from extracted text is missing these marks, it cannot be reliably reconstructed by pattern matching. The name Ọ̀ṣun becomes Osun; Àràbà becomes Araba. Both could represent different words with different meanings.

> **WARNING:** *Never attempt to add diacritics to Yoruba names programmatically using a lookup table or rule system. Yoruba is a tonal language. Adding the wrong tone mark creates a different word. All Yoruba name corrections must be made by a human with Yoruba linguistic knowledge, verified against the print source.*

The practical approach for the Verger dataset: extract what the PDF contains, flag all Yoruba name fields as requiring diacritic review, and maintain a separate review queue for corrections. For the LS563 project, diacritical marks were provided manually drawing on the Executive Director's knowledge of Yoruba orthography and cross-reference with Verger's print edition.

Family name truncation

Botanical family names in the Verger glossary sometimes appear on a continuation line or get cut off at a line break. A family like "Bombacaceae" may be truncated as "Bom-" with the rest on the next line. Build a continuation check into your parser that looks ahead one line when a family name appears incomplete.

Page header contamination

Running page headers get extracted as part of the text stream. In the Verger glossary, the phrase "Glossário científico-iorubá" appeared intermittently throughout extracted text. These must be filtered before parsing:

```
FILTER_LINES = [
    'Glossário científico-iorubá',
    'Glossário iorubá-científico',
    'Guide to Afro-Cuban Herbalism',
    'Dalia Quiros-Moran',
]

lines = [l for l in raw_lines
         if not any(f in l for f in FILTER_LINES)
         and not l.strip().isdigit()   # filter page numbers
]
```

Step 3.5 — The pipe delimiter: the no-space rule

Multiple values within a single cell are separated by the pipe character (|). This is the convention used throughout the dataset, and it connects directly to OpenRefine's "Split multi-valued cells" function in the transformation pipeline.

> **RULE:** The pipe delimiter is always written without surrounding spaces: Iròkò|Àràbà|Egigun, never Iròkò | Àràbà | Egigun. This rule is permanent and applies to every pipe-delimited column in every dataset produced by this workflow.

The reason this rule exists: the space-around-pipe convention was tried during the initial dataset assembly and caused mismatches in OpenRefine's split function, which splits on the literal delimiter character. A pipe surrounded by spaces creates three separate tokens — the name, a space, and another space — rather than two clean name values. Removing the spaces fixed the mismatches. The rule is enforced by the VBA utilities documented in Part 5.

Step 3.6 — Recognizing when a parsing approach is not working

These are the signals that you need to stop, reassess the approach, and try something different rather than continuing to refine the current parser:

- Output record count is less than 40% of expected after three or more fix iterations.
- The same field is producing a mix of correct values and obviously wrong values (family names appearing as Yoruba names, for example).
- Entries from different plants are being merged into a single record.
- The parser produces correct output for one section but completely fails on another section that appears structurally similar.

When any of these occur, go back to raw pdftotext output for the failing section, look at it line by line, and understand the actual format before writing more code. The problem is almost always in the source text structure, not in the parsing logic itself.

Part 4: Cross-Referencing the Two Sources

Why scientific name is the only reliable join key

The two sources use different naming conventions in every language. The Spanish name in Quiros-Moran for Ceiba pentandra may not match any name Verger uses for the same plant. Yoruba names vary by dialect, orthography, and regional convention. English names have no standardization. Scientific binomial names (Genus species) are the only field consistent enough across sources to serve as a join key.

Even here there are complications: taxonomic synonymy means the same plant may appear under different accepted names in different sources depending on publication date. The cross-reference strategy was:

6. Extract the complete list of scientific names from Verger's glossary.
7. Extract the complete list of scientific names from Quiros-Moran.
8. Find the intersection — plants present in both sources.
9. For plants that do not match on the primary name, attempt synonym matching using GBIF or Global Names Verifier.
10. For any remaining non-matches, assess manually using domain knowledge.

Building the master plant list

The initial plant list was seeded from Verger's scientific glossary. For the LS563 project, a subset of 50 plants was selected from the full glossary based on:

- Significance to the Lucumi/Yoruba ritual pharmacopeia — practitioner selection criteria.
- Presence in both Verger and Quiros-Moran — ensures cross-reference richness.
- Taxonomic diversity — representing multiple families and ecological contexts.
- Geographic distribution — African species, Caribbean naturalized species, pantropical.

> **WARNING:** *Interns should not make independent decisions about which plants to include or exclude from the dataset. The selection criteria are not purely botanical. Which plants have sacred significance and which belong to which Orisha requires initiation-level knowledge to adjudicate correctly. All scope decisions must be approved by the Executive Director.*

The name collision problem

Many Yoruba plant names are not exclusive to a single species. The name Ìṣín is attested for both Blighia sapida (akee tree) and Aloe spp. in different contexts. The name Ìròkò is used for both Milicia excelsa (the true African iroko tree) and Ceiba pentandra in different regional and diaspora contexts. This is not an error in either source. It reflects the lived reality of a knowledge system that evolved across generations, geographies, and communities.

For linked data purposes, the Yoruba name alone cannot reliably identify a species. The relationship between a Yoruba name and a scientific name is many-to-many, not one-to-one. The dataset handles this with two fields:

- **Name Collision (Yes/No):** Boolean flag indicating whether the primary vernacular name is shared with another species in the dataset.
- **Collision Notes:** Free text explaining which other species shares the name and in what context.

These fields were populated manually during dataset assembly and require practitioner knowledge to fill in correctly.

Part 5: Assembling the Excel Dataset

Column structure

The dataset uses a fixed column structure. This structure was finalized after several iterations and must not be changed without updating the mapping workbook (Volume 3), the controlled vocabulary mappings, and the OpenRefine transformation skeleton (Volume 2) simultaneously. Adding a new column is a coordinated change across all three documents.

The full column structure is:

Column Name	Language Tag	Data Type	Notes
Plant ID	—	Text	Sequential: Plant0001, Plant0002... Padded to 4 digits. Never reused once assigned.

Column Name	Language Tag	Data Type	Notes
Scientific Name	—	Text	Accepted binomial. Verified against GBIF or Catalogue of Life. No author citations or synonyms.
Plant Synonyms	—	Pipe-delimited	All known synonyms. No spaces around pipe.
Yoruba Name(s)	@yo	Pipe-delimited	All Yoruba names from Verger's glossary. Requires diacritic review against print source.
Primary Vernacular Name	—	Text	Most commonly used name in Cuban practice (Quiros-Moran). Used as skos:prefLabel.
Name (English)	@en	Text	English common name from Quiros-Moran or botanical sources.
Name (Cuba)	@es	Pipe-delimited	Cuban Spanish name(s) from Quiros-Moran.
Name (Lucumi)	@x-lucumi	Pipe-delimited	Lucumi names from Quiros-Moran. Private-use BCP 47 language tag.
Name (Haiti)	@ht	Pipe-delimited	Haitian Creole vernacular names from Quiros-Moran.
Name (Vodou)	None	Pipe-delimited	Haitian Vodou ritual/liturgical names. No language tag — liturgical language varies by entry.
Name (Brazil)	@pt-BR	Pipe-delimited	Brazilian Portuguese vernacular names. Populated from the Portuguese manuscript.
Name (Candomblé)	None	Pipe-delimited	Candomblé ritual/liturgical names. No language tag — may be Nagô, Bantu, or Portuguese.
Other Vernacular Names	—	Pipe-delimited	Country-prefixed names: Colombia, Mexico, Venezuela, Haiti, etc. Format: Country - Name. No spaces around pipe.
Name Collision (Yes/No)	—	Boolean text	"Yes" if primary Yoruba name is shared with another species. "No" otherwise.
Collision Notes	—	Text	Which other species shares the name and in what context.
Medicinal Uses	—	Controlled vocab	One of four values. See Part 5 controlled vocabulary section.
Access Level	—	Controlled vocab	One of six values. Requires Executive Director approval. See Volume 3.
Ritual Use	—	Controlled vocab	One of seven values. Requires practitioner judgment.
Ritual Notes	—	Text	Free text annotation on ritual context from either source or practitioner knowledge.

> **NOTE:** *The Name (Vodou) and Name (Candomblé) columns have no language tag by design. Vodou liturgical language can be Haitian Creole, Fon-Gbe derived, or liturgical French depending on the nation and ceremony. Candomblé liturgical language can be Nagô/Yoruba-derived, Bantu-derived, or Portuguese depending on the nation. Forcing a single language tag onto these fields would be false precision. They are plain text literals in the RDF output.*

The Other Vernacular Names convention

Regional vernacular names from Quiros-Moran that do not have their own dedicated column are folded into the Other Vernacular Names field using a country-prefix convention. The format is:

```
Colombia - Nombre1|Mexico - Nombre1|Venezuela - Nombre1

# Multiple values from the same country:
Colombia - Nombre1|Colombia - Nombre2|Mexico - Nombre1

# NOT this (spaces around pipe):
Colombia - Nombre1 | Mexico - Nombre1    ← WRONG
```

Country labels must be used consistently across all records. Use the full country name, not abbreviations. The authorized country label list for Other Vernacular Names is: Colombia, Costa Rica, El Salvador, Guatemala, Honduras, Mexico, Nicaragua, Panama, Puerto Rico, US, Venezuela. If a future source adds a country not in this list, add it to the list in Volume 3 (the mapping workbook documentation) before populating the dataset.

> **NOTE:** *Haiti has its own dedicated column (Name (Haiti)) and does not appear in Other Vernacular Names. Brazil has its own dedicated column (Name (Brazil)) and does not appear in Other Vernacular Names. Cuba and Lucumi likewise have dedicated columns. Only the remaining regional variants from the Quiros-Moran country list go into Other Vernacular Names.*

Pipe-delimited fields and how to populate them

Several columns contain multiple values for a single plant record. These use the pipe character (|) as a delimiter within a single cell, rather than separate rows or columns. This allows the record to remain in a single row while preserving all values, and connects directly to OpenRefine's split function in the transformation pipeline.

Rules for populating pipe-delimited fields:

- Put the most authoritative or most commonly used value first.
- No spaces around the pipe: Iròkò|Àràbà, never Iròkò | Àràbà.
- No trailing pipe at the end of the list.

- No double pipes (||) anywhere in the field.
- If there is only one value, write it without any pipe character.
- Empty cells must be truly empty — no pipe, no space, no placeholder text.

Excel VBA utilities for pipe-delimited data

Three VBA functions were used during dataset assembly to manage pipe-delimited fields. These are stored in the Excel workbook's VBA module. To access them, open the Visual Basic Editor with Alt+F11, insert a new module, and paste the code below.

MergeWithPipe — combine a range of cells into one pipe-delimited string

When extracting data from multiple source columns into a single pipe-delimited cell, this function lets you write a formula such as

`=MergeWithPipe(B2:F2)` rather than manually concatenating each cell. It automatically skips blank cells, so you do not get leading or trailing pipes when some columns are empty.

```
Function MergeWithPipe(rng As Range) As String
    Dim cell As Range
    Dim result As String

    result = ""
    For Each cell In rng
        If cell.Value <> "" Then
            If result = "" Then
                result = cell.Value
            Else
                result = result & "|" & cell.Value
            End If
        End If
    Next cell

    MergeWithPipe = result
End Function
```

RemoveTrailingPipe — clean up stray trailing pipes

When data is assembled from multiple extraction passes or entered quickly, cells sometimes end up with a trailing pipe (e.g., "Iròkò|Àràbà|"). Select the affected column or range and run this sub from the Developer ribbon or the VBA editor.

```
Sub RemoveTrailingPipe()
    Dim c As Range
    For Each c In Selection
        If Right(c.Value, 1) = "|" Then
            c.Value = Trim(Left(c.Value, Len(c.Value) - 1))
        End If
    Next c
End Sub
```

RemoveDoublePipes — fix double-pipe artifacts from blank cells

When MergeWithPipe is applied to a range that contains blank cells in the middle, the sequence "| |" (pipe-space-pipe) can appear where a blank cell was skipped but a space character remained. This sub cleans those artifacts. It runs in a loop until no double-pipe sequences remain.

> **NOTE:** *Run RemoveTrailingPipe before RemoveDoublePipes. If a trailing pipe is present when RemoveDoublePipes runs, the Do While loop may not terminate correctly on certain edge cases.*

```
Sub RemoveDoublePipes()
    Dim c As Range
    For Each c In Selection
        Do While InStr(c.Value, "| |") > 0
            c.Value = Replace(c.Value, "| |", "|")
        Loop
    Next c
End Sub
```

The recommended sequence when assembling any pipe-delimited column: (1) apply MergeWithPipe as a formula to combine source cells, (2) paste-as-values to replace the formula with static text, (3) run RemoveTrailingPipe on the column, (4) run RemoveDoublePipes on the column.

The controlled vocabulary columns

Three columns use a controlled vocabulary from the Iroko Framework: Medicinal Uses, Access Level, and Ritual Use. The values in these columns must match exactly the human-readable labels defined in the controlled vocabulary. These labels are later mapped to Iroko concept slugs during the OpenRefine transformation phase (see Volume 2). The full mapping is documented in Volume 3.

Medicinal Uses (4 values)

- Digestive support
- General tonic / revitalizing
- Respiratory support
- Skin and topical use

Access Level (6 values)

- Public - Unrestricted
- Public - No Amplification
- Limited - Community Only
- Limited - Initiated Only
- Restricted - No Access

- Private - Practitioner Access

> **WARNING:** *Access Level assignments require practitioner judgment and Executive Director approval. An intern must never assign an Access Level independently. All access level entries must be reviewed and approved by the Executive Director before the dataset enters the transformation pipeline. When in doubt, assign a more restrictive level and flag the record for review. The full decision protocol is documented in Volume 3.*

Ritual Use (7 values)

- Cosmological or Symbolic Association
- Healing and Restoration
- Invocation and Communication
- Offering and Devotion
- Protection and Boundary Setting
- Purification and Cleansing
- Rites of Transition

Plant ID assignment rules

Plant IDs are assigned sequentially in the format Plant0001, Plant0002, etc., padded to four digits. Once assigned, a Plant ID is never reused, reassigned, or renumbered, even if a record is removed from the active dataset. Removed records are flagged as inactive rather than deleted so that the ID sequence remains traceable. When adding records from a new source, continue the sequence from the highest existing ID.

Part 6: How to Work with Claude on PDF Parsing Tasks

What Claude can do well

- Writing Python extraction scripts once the entry format has been described clearly.
- Diagnosing why a specific parser is missing records, given sample output.
- Writing GREL or Python expressions for data cleaning and standardization.
- Cross-checking extracted data against a known list (verifying that expected scientific names are all present).
- Building the Excel workbook structure and populating it from a parsed output file.
- Explaining error messages from extraction tools.

What Claude cannot do

- **Claude cannot read a PDF from a previous session.** If you are continuing a parsing project, you must re-upload the relevant files or share the relevant data again. Claude has no memory of files from previous conversations.
- **Claude cannot see images in a PDF.** If the PDF is a pure image scan with no embedded text layer, Claude cannot extract any text. OCR must be run first using a tool like Tesseract or Adobe Acrobat.
- **Claude cannot add Yoruba diacritics.** Adding or correcting tone marks requires linguistic knowledge that Claude does not have at the level required for scholarly accuracy.
- **Claude cannot make Access Level or Ritual Use assignments.** These decisions require practitioner knowledge and are not delegatable to any AI system.
- **Claude may produce code that silently drops records.** Always count output records and compare against expected counts. Never assume a script that runs without errors is also producing complete output.

Best practices for a parsing session with Claude

11. **Describe the document structure in words first** before asking for code. Tell Claude: the PDF is a scanned two-column text; the glossary runs from pages 615 to 720; entries follow the format [Yoruba name] -[SCIENTIFIC NAME], [Family]. The more specific you are, the better the code will be.
12. **Always share sample extracted text.** Run pdftotext on a 5–10 page section and paste the output into the conversation. Claude writes far better parsers when it can see the actual text format rather than guessing.
13. **Test on small samples first.** Ask Claude to write the parser so it runs on pages 615–620 only. Verify it works on that sample before expanding to the full document.
14. **Ask Claude to include a missing-record diagnostic.** Any parsing script should include a step that counts output records and prints the first few entries for manual inspection.
15. **Share specific failure cases.** When entries are missing, paste the exact text from the PDF that was not captured. Claude can then diagnose the regex or parsing logic failure and fix it.
16. **Do not assume the first working script is final.** Ask Claude: "What kinds of entries might this script miss?" This surfaces problems proactively.
17. **Keep a session summary.** After a successful parsing session, ask Claude to write a brief summary of what the script does, its known limitations, and what would need to change for a differently formatted source. This becomes documentation for the next person.

Part 7: Integrating Additional Source Texts

Overview

The current dataset draws on two sources: Verger and Quiros-Moran. Future work will bring in additional sources including a Brazilian Portuguese botanical manuscript, Lydia Cabrera's El Monte (via the David Font translation), fieldwork data from Babaláwo Irete Obara in Matanzas, and potentially other texts. This part documents how to integrate a new source into the existing dataset rather than starting a new one.

The central principle: the dataset is a single unified record per plant species. Adding a new source means enriching existing records with new fields and values, not creating parallel datasets. Every new source must go through the same scientific name cross-reference process documented in Part 4 before a single cell is populated.

Step 7.1 — Characterize the new source before parsing

For each new source text, answer these questions before beginning:

- What languages and traditions does it cover? Does it add content to existing columns, or does it require new columns?
- What is its physical format — digital native PDF, scanned print, manuscript scan, database export, fieldwork notes?
- What is its primary identifier for plant entries — scientific name, vernacular name, page number, entry number?
- What is the scan quality if it is a scanned document? Are diacritics preserved?
- Does it use the same scientific name standard as the existing dataset (GBIF backbone, Catalogue of Life), or does it use older synonyms that need reconciliation?

Do not begin parsing until these questions are answered. The answers determine the tools, the parsing strategy, and what new columns (if any) are needed.

Step 7.2 — The Brazilian Portuguese manuscript

The Brazilian Portuguese manuscript is a priority next source. It populates two columns that are currently empty in the dataset: Name (Brazil) and Name (Candomblé). Both columns are already in the schema — no structural change to the dataset is required.

Key considerations specific to this source:

- **Nagô orthography.** Candomblé Ketu/Nagô ritual names are derived from Yoruba but written in Brazilian Portuguese orthographic conventions. The same plant name may look different from Verger's Yoruba spelling. Do not attempt to normalize these to standard Yoruba orthography — record them as they appear in the source.
- **Nation variation.** Candomblé has multiple nations (Ketu, Angola, Jeje, etc.) with different ritual vocabularies. If the manuscript identifies the nation, record it in the Ritual Notes field for that record.
- **Language tag.** Name (Brazil) takes the tag @pt-BR. Name (Candomblé) takes no language tag, consistent with the no-tag rule for liturgical columns.

- **Access level.** Candomblé ritual names carry the same access level considerations as Lucumi and Vodou names. All access level assignments require Executive Director approval.

Step 7.3 — Haitian Vodou sources

The Name (Haiti) and Name (Vodou) columns are already in the schema. Future Haitian sources populate these columns. Key considerations:

- **Haitian Creole orthography.** Standard Haitian Creole uses a consistent phonemic orthography (SIK). Record names as they appear in the source. If the source predates standardized HC orthography, note this in Ritual Notes.
- **Vodou nation variation.** Haitian Vodou has multiple rites (Rada, Petwo, Gede, Kongo, etc.) with different plant associations and names. If the source identifies the rite, record it in Ritual Notes.
- **No language tag on Vodou column.** Vodou liturgical language can be Haitian Creole, Fon-Gbe derived, or liturgical French. The no-tag rule applies.

Step 7.4 — Cabrera's El Monte

Lydia Cabrera's El Monte (via the David Font translation) covers Afro-Cuban sacred plant knowledge with significant overlap with Quiros-Moran but with deeper ritual detail. Before integrating this source, note:

- **El Monte blurs the non-operational boundary.** The text goes into ritual detail beyond what the current dataset schema is designed to hold. Data from El Monte should be used to enrich existing columns (Ritual Notes, Collision Notes) rather than to create operational ritual content in structured fields.
- **Lucumi names.** Cabrera is a primary authority for Lucumi plant names. Where Cabrera's Lucumi name differs from Quiros-Moran's, add Cabrera's variant to the Name (Lucumi) pipe-delimited field with a note in Ritual Notes identifying the source of each variant.
- **Scientific name reconciliation.** Cabrera's botanical identifications are not always consistent with modern taxonomy. Verify all scientific names against GBIF or Catalogue of Life before accepting them as join keys.

Step 7.5 — Fieldwork data from Irete Obara

Fieldwork data from Babaláwo Irete Obara in Matanzas represents a different category of source: oral transmission rather than published text. The following rules govern its integration:

- Oral attestations populate the same columns as published sources but must be flagged as fieldwork data in the Ritual Notes field, with the date, location, and name of the source (Babaláwo Irete Obara, Matanzas, [date]).
- Oral ritual names for Lucumi, Vodou, or Candomblé columns require Executive Director review before entry, regardless of other access level assignments.

- Where fieldwork data conflicts with published sources, both values are preserved. The fieldwork value is added to the pipe-delimited field. The conflict is documented in Collision Notes.
- Scientific name identifications from fieldwork require botanical verification against GBIF before being used as join keys. A practitioner's identification of a plant by Yoruba name is authoritative on the Yoruba name, not necessarily on the Linnaean binomial.

Step 7.6 — Merge logic for existing records

When a new source provides data for a plant already in the dataset, update the existing record rather than creating a new one. The merge rules are:

- **Pipe-delimited fields:** Append new values to the existing pipe-delimited string if the value is not already present. Do not duplicate values. Run RemoveDoublePipes and RemoveTrailingPipe after any programmatic merge operation.
- **Controlled vocabulary fields:** Do not overwrite an existing value without Executive Director approval. If the new source suggests a different Medicinal Use or Ritual Use category, document the discrepancy in Ritual Notes and flag the record for review.
- **Access Level:** A new source can only raise the access level of a record (make it more restrictive), never lower it, without explicit Executive Director approval.
- **Collision detection:** Before merging any vernacular names from a new source, check whether the new source introduces a name already used for a different species in the dataset. If so, update the Name Collision flag and Collision Notes for both affected records.
- **Plant ID:** Never reassign or change Plant IDs during a merge. If a new source introduces a plant not in the dataset, assign the next available Plant ID and create a new record.

Appendix A: Quick Reference Commands

Essential pdftotext commands

```
# Extract all text from a PDF
pdftotext yourfile.pdf output.txt

# Extract specific pages
pdftotext -f 615 -l 720 yourfile.pdf -

# Layout-preserving extraction
pdftotext -layout -f 296 -l 296 yourfile.pdf -

# Search extracted text for a known string
pdftotext yourfile.pdf - | grep -i 'ceiba pentandra'

# Count lines in extracted text
```

```
pdftotext yourfile.pdf - | wc -l

# Find what line a string appears on
pdftotext yourfile.pdf - | grep -n 'CEIBA'

# View lines around a match
pdftotext yourfile.pdf - | grep -B5 -A5 'CEIBA'
```

Essential Python patterns

```
import pdfplumber
import fitz            # PyMuPDF
import re

# Open and inspect with pdfplumber
with pdfplumber.open('yourfile.pdf') as pdf:
    print(f'Pages: {len(pdf.pages)}')
    text = pdf.pages[295].extract_text()    # 0-indexed
    print(text[:500])

# Open and inspect with PyMuPDF
doc = fitz.open('yourfile.pdf')
page = doc[295]                             # 0-indexed
print(page.get_text()[:500])

# Strip leading non-alphabetic junk (OCR corruption)
clean = re.sub(r'^[^A-Za-z\u00c0-\u024f]+', '', raw_string)

# Filter known page headers and page numbers
HEADERS = ['Guide to Afro-Cuban', 'Glossário científico']
lines = [l for l in raw.split('\n')
         if not any(h in l for h in HEADERS)
         and not l.strip().isdigit()]

# Count output and check against expected
print(f'Extracted: {len(results)} records')
print(f'Expected:  ~273')
if len(results) < 200:
    print('WARNING: significant records may be missing')
```

Authorized country labels for Other Vernacular Names

```
# Use exactly these labels in the Country - Name prefix format
COUNTRIES = [
    'Colombia',
    'Costa Rica',
    'El Salvador',
    'Guatemala',
    'Honduras',
    'Mexico',
    'Nicaragua',
    'Panama',
```

```
    'Puerto Rico',
    'US',
    'Venezuela',
]

# Haiti and Brazil have dedicated columns — do NOT prefix them here
# Cuba and Lucumi have dedicated columns — do NOT prefix them here
```

Appendix B: Production QC Checklist

This checklist must be completed and signed off by the Executive Director before any dataset version is advanced to the OpenRefine transformation pipeline. Keep a completed copy on file for each dataset version.

Section 1: Record Count and Completeness

Check	Pass	Fail
Total record count matches expected scope (document before beginning)	☐	☐
No Plant ID is duplicated in the dataset	☐	☐
No Plant ID gap in the sequence (no skipped numbers)	☐	☐
Every record has a Scientific Name	☐	☐
Every record has a Plant ID	☐	☐
Every record has an Access Level value	☐	☐
Every record has a Ritual Use value or a note explaining absence	☐	☐

Section 2: Pipe Delimiter Compliance

Check	Pass	Fail
No spaces around any pipe delimiter in any column (search for " | " across all cells)	☐	☐
No trailing pipes in any cell (RemoveTrailingPipe has been run)	☐	☐
No double pipes in any cell (RemoveDoublePipes has been run)	☐	☐
Other Vernacular Names entries use the authorized Country - Name format	☐	☐

Check	Pass	Fail
Country labels in Other Vernacular Names match the authorized list exactly	☐	☐

Section 3: Controlled Vocabulary Compliance

Check	Pass	Fail
All Medicinal Uses values exactly match one of the four authorized values	☐	☐
All Access Level values exactly match one of the six authorized values	☐	☐
All Ritual Use values exactly match one of the seven authorized values	☐	☐
No unauthorized or variant-spelling values in any controlled vocabulary column	☐	☐
All controlled vocabulary values are verified against the mapping workbook (Volume 3)	☐	☐

Section 4: Diacritic and Language Review

Check	Pass	Fail
Yoruba Name(s) column has been reviewed against Verger print source for tone marks	☐	☐
All Yoruba names requiring diacritic correction have been flagged or corrected	☐	☐
Name (Lucumi) values have been verified by the Executive Director	☐	☐
Name (Vodou) values have been verified by the Executive Director	☐	☐
Name (Candomblé) values have been verified by the Executive Director	☐	☐
No diacritic review is pending for records entering this pipeline run	☐	☐

Section 5: Access Level and Ritual Use Sign-Off

Check	Pass	Fail
All Access Level assignments have been reviewed by the Executive Director	☐	☐
All Ritual Use assignments have been reviewed by a practitioner	☐	☐

Check	Pass	Fail
All Name (Vodou) entries have been reviewed for access level appropriateness	☐	☐
All Name (Candomblé) entries have been reviewed for access level appropriateness	☐	☐
All Name (Lucumi) entries have been reviewed for access level appropriateness	☐	☐
Collision Notes have been completed for all records where Name Collision = Yes	☐	☐

Section 6: Scientific Name Verification

Check	Pass	Fail
All Scientific Name values are accepted binomials (no author citations, no synonyms)	☐	☐
All scientific names have been verified against GBIF or Catalogue of Life	☐	☐
Plant Synonyms field is populated from GBIF or Global Names Verifier where available	☐	☐
Any scientific names from fieldwork data have been botanically verified	☐	☐
I/J OCR confusion has been checked for any names that failed initial reconciliation	☐	☐

Sign-Off

Dataset Version	Date	Executive Director
Notes:		

> **NOTE:** *A completed copy of this checklist must be retained for each dataset version that enters the transformation pipeline. File as: QC_Checklist_[DatasetName]_[Version]_[Date].pdf*